

CSP

Constraint Satisfaction Problems:

• state = set of variables with values.

Solution = each variable has a value that satisfies all the constraints on the variable!

↓
uses general purpose heuristics.

rather than problem-specific to solve complex problems.

✦ Its definition

□ X : set of variables $\{x_1, \dots, x_n\}$.

□ D : set of domains $\{D_1, \dots, D_n\}$
← each domain

values of variable x_i : $D_i = \{v_1, \dots, v_k\}$

□ C : set of constraints, specifies the allowable combinations of values.

↓
 $\langle \text{scope}, \text{rel} \rangle$ rel: is the relation that defines the values.
• tuple of variables that participate in the constraint

A state: is defined by

an assignment of values to some or all of the variables
 $\{x_i = v_i, x_j = v_j, \dots\}$.

①. legal assignment: does not violate any constraints
also called Consistent

②. Complete assignment: every variable is assigned.

③. partial assignment: assign values to only some of the variables.

Solution: is a Consistent Complete & assignment.

relation representation:

• explicit list of all types of values that satisfy the constraint

• Abstract relation supports 2 operations

→ test if a tuple is a member of the relation

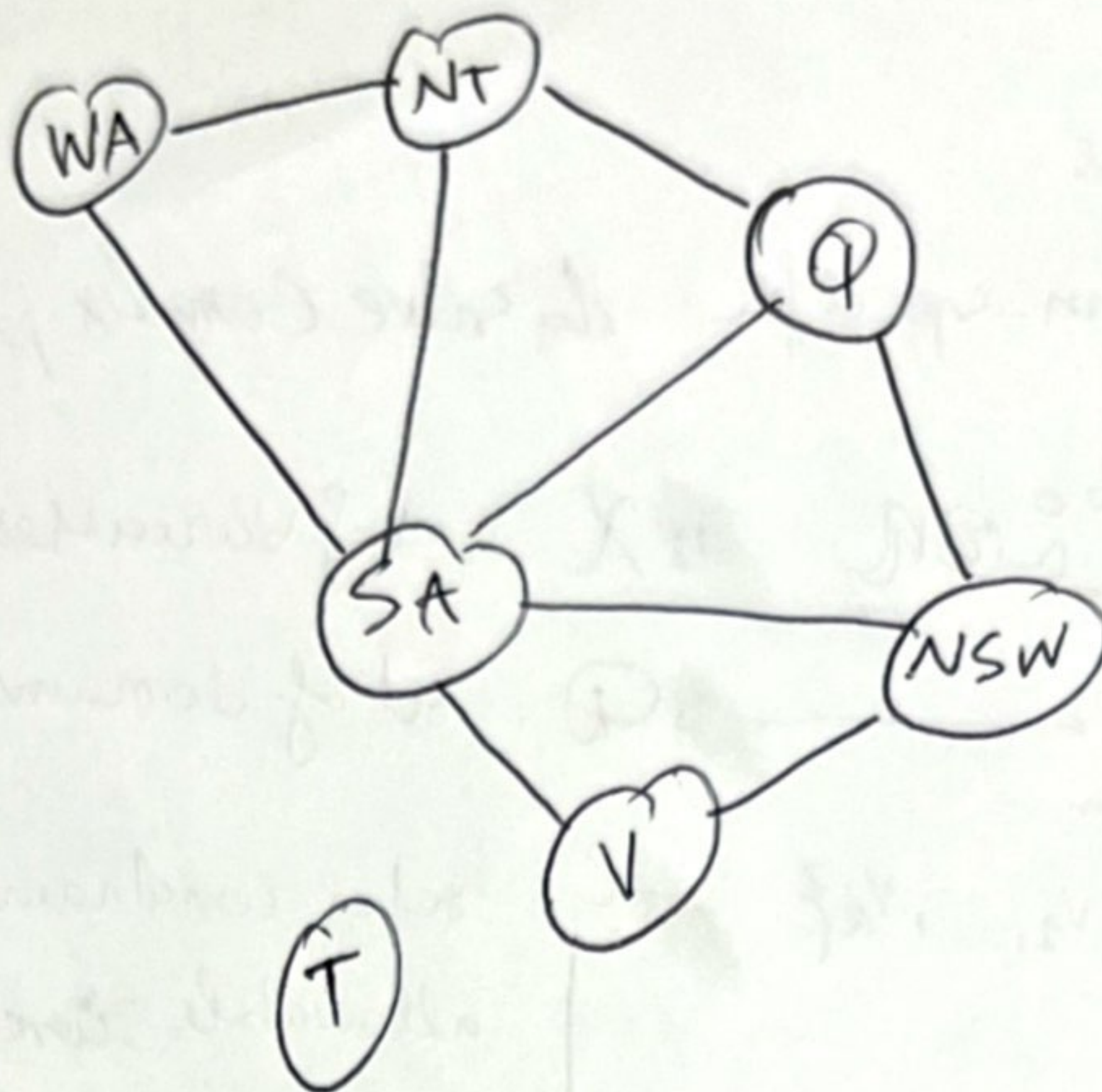
→ enumerating the members of the relation.

they can eliminate large parts of the search space.

• Possible Visualization of the problem: Constraint Graph

→ nodes are problem variables

→ links are any 2 variables that participate in a constraint



Remark:

CSP can reduce upto 87% of assignments compared to other searches

Note that: problem can be solved quickly with CSP.

Difference is when deciding a state is a goal:

- Partial assignment is found:

→ discard further refinements

→ see why the assignment is not a solution? Which variables violate a constraint

focus on them!

slide 13

♦♦ types of constraints:

Can be solved with optimization
Search methods either path-based or local.
= Constraint optimization problem.

① Precedence constraints: lead to the optimal solution (tasks before tasks). - Can be encoded as costs on individual variable assignment.

② Disjunctive constraint: two variables must not have the same trip at the same time

$$\begin{aligned}
 & \text{A} \times \text{F} + 20 \leq \text{A} \times \text{B} \\
 & \text{A} \times \text{B} + 10 \leq \text{A} \times \text{F}
 \end{aligned}$$

Simplest CSP?? → variables have

discrete / finite / domains

Can be infinite as if no dead end for example.

Case of ∞ domains: - Constraints cannot be described by enumerating all combinations

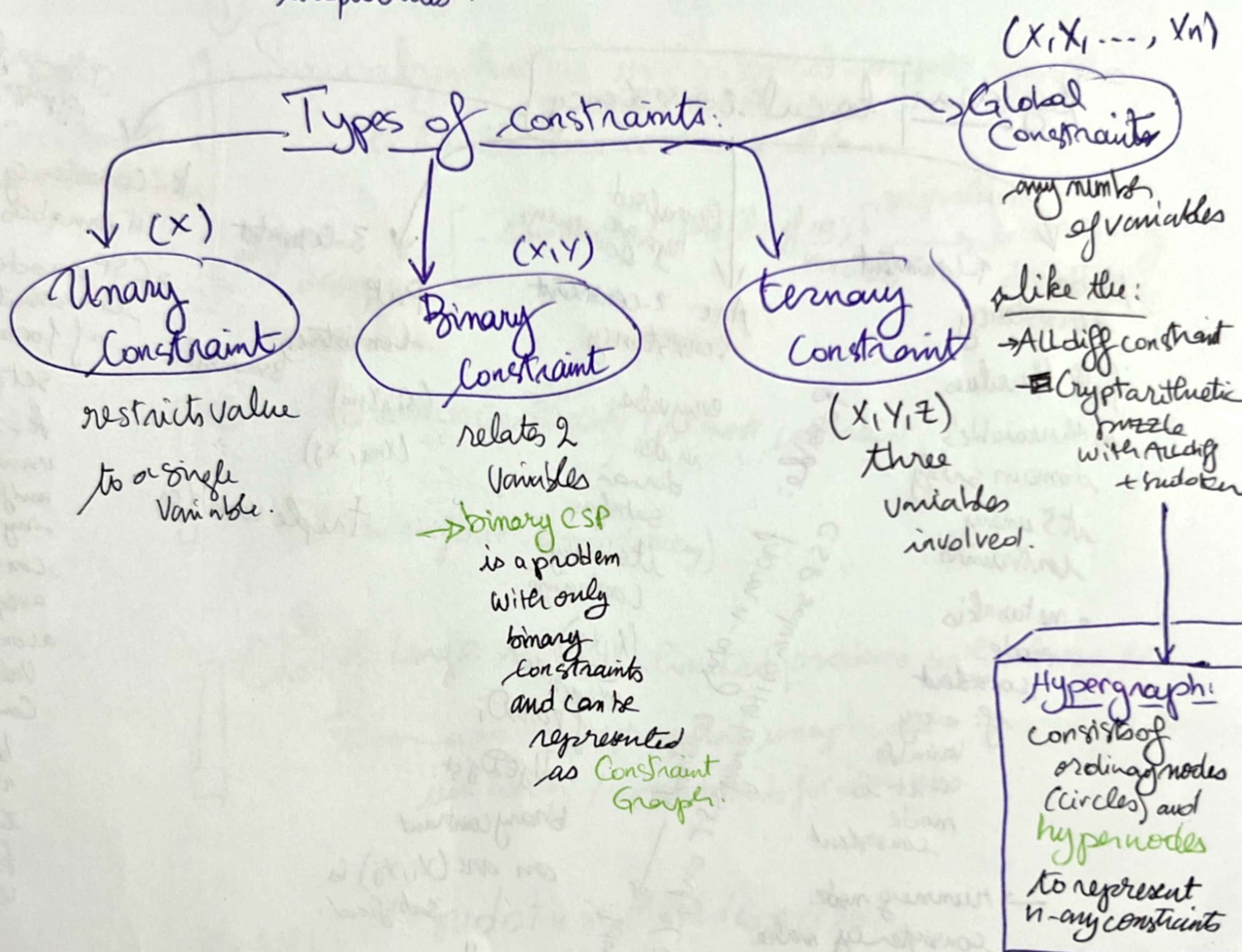
Example:

CSPs of linear programming problems.

→ constraints must be: linear equations or inequalities.

- Need for Constraint language to express a constraint like

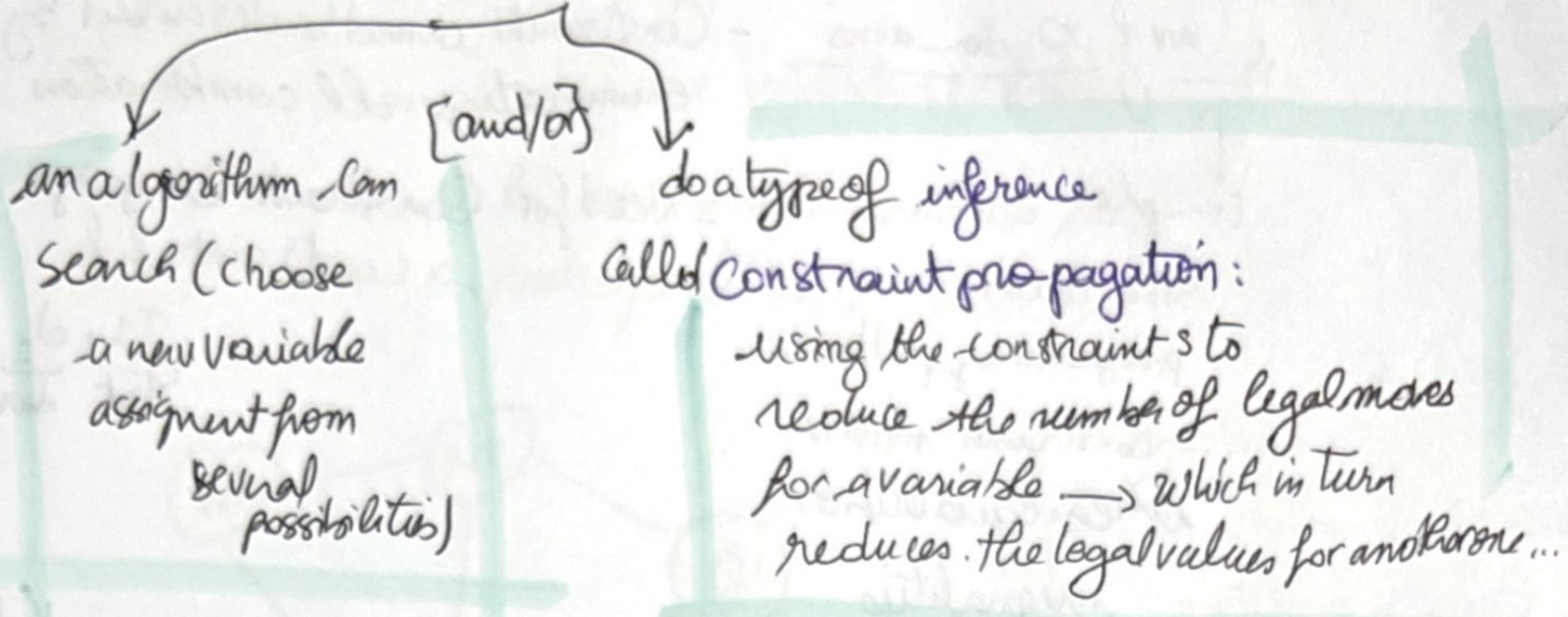
$$\frac{T_1 + d_1}{\text{start duration}} \leq T_2^{\text{start}}$$



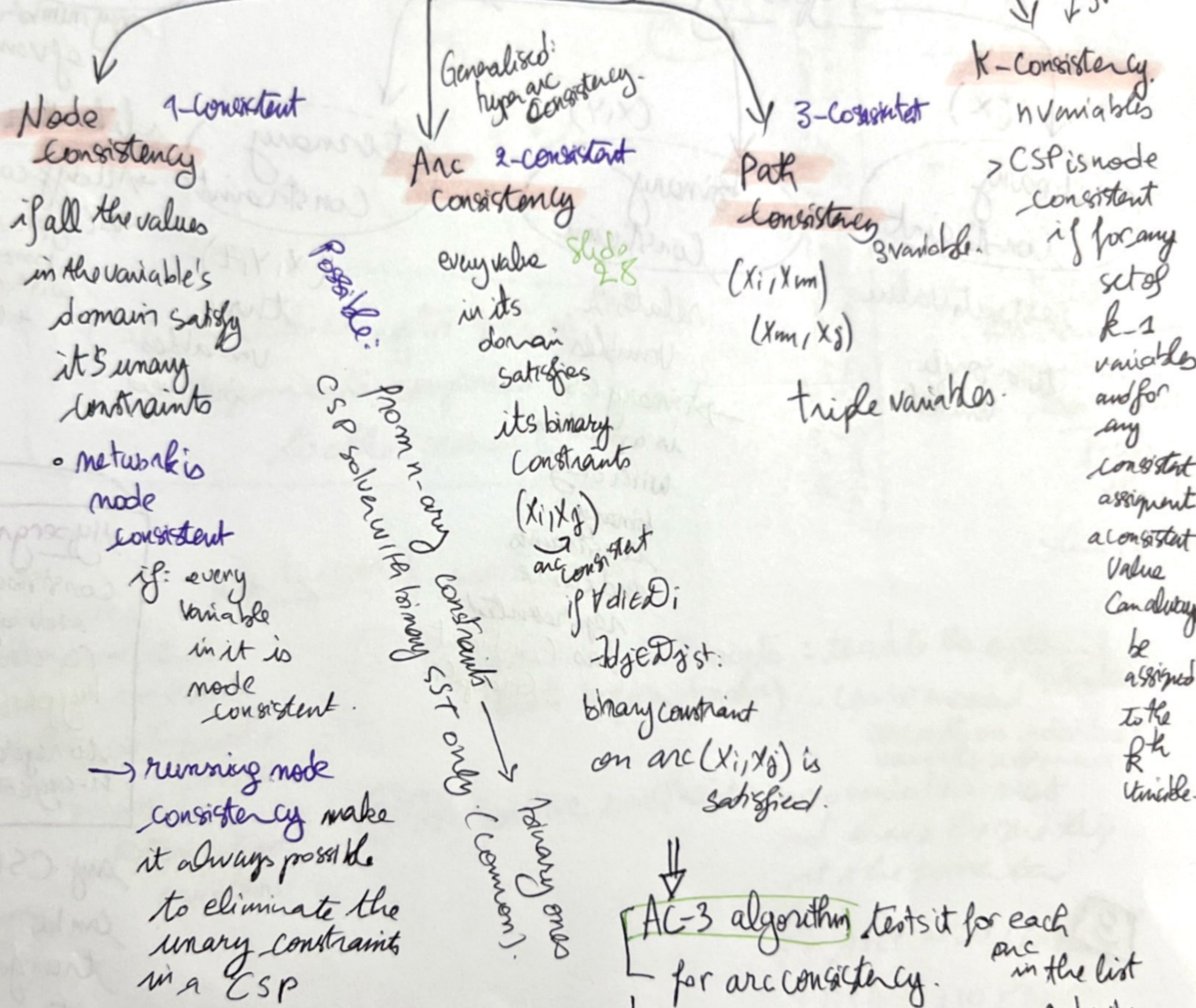
any CSP can be transformed into one with only binary constraints

variable = n in the network

We have a choice in CSP



Key idea: local consistency



AC-3 algorithm tests it for each arc in the list and when we change some procedures may be taken for x_i

case of failure

- if D_i is revised to the $\emptyset$
- $\rightarrow$ failure returned

Slide 30

* Strongly K -Consistent CSP: if it is k consistent and also
↓
($k-1$) consistent and ($k-2$)... all the way
down to 1-consistent.

Guaranteed to find solution in $O(n^2d)$

→ however, any such algorithm must take time exponential
in the worst case, same for space complexity.

Global Constraints

- ① Some algorithms are used as special-purpose algorithms to handle some constraints like:
algorithm inside 38. ← the one that checks if $m > n$ ^{set of values} → inconsistency detected!

- ② Resource constraint / Atmost constraint
(Can enforce consistency).

- ③ Case of Large resource-limited problems with values $\in \mathbb{N}$:
Domains are represented using bounds propagation -
use lower / upper bounds for domains.

EXAMPLE: Sudoku in the slides till 46.

→ Strategy: "Naked triples" (new tricks that we can remove some values).

Backtracking Search for CSPs

- When Cannot be solved using inference alone.
- Use of depth-limited search.
- State = partial assignment and action is: add $var = value$ to the assignment.

Complexity is simplified when considering that CSPs are "commutative" (order of application of any given set of actions has no effect on the outcome)

→ With this restriction: we get d^n leaves

Backtracking Search: a DFS that chooses values for one variable at a time and backtracks when a variable has no legal values left to assign.

→ the ALGORITHM: keeps only a single state and alters that representation rather than creating new ones as in classical search.

some notes
still need
to be added
here.

★ Improving the CSP Solving efficiency:

- unlike classical search where domain specific heuristics were used
- CSPs can be solved without such domain specific knowledge.

① Variable Ordering: (which variables to choose to assign???)

★ Select unassigned-variable: choose the next unassigned variable in order $\{x_1, \dots, \dots\}$.
(Not the most efficient)

← Sometimes do not help (like when all are same)
★ Minimum Remaining-Values (MRV): variable with the fewest legal values.
(heuristic).
(better than random static ordering)
(more powerful guide)
fail-first / most constrained variable.

★ Degree Heuristic: to reduce the branching factor by selecting the variable involved in the largest number of constraints on other unassigned variables.
(useful as a tie-breaker)

② Value Ordering: which value to choose??

★ Least-Constrainting-Value: heuristic:
values that rules out the fewest choices for neighboring variables in the graph → to leave the maximum flexibility for subsequent variable assignments.

7

Interleaving Search and inference.

• Forward Check: type of inference.

- uses arc-consistency.

→ delete from other domains the value that
| other
| variables
| that were not
| assigned
| the values that
| ~~do~~ not satisfy
| the constraint

• MAC: maintaining arc consistency.

Intelligent Backtracking:

Looking
backward

• Chronological backtracking
algorithm seen previously.

→ fails → back up to the
preceding variable.

(sometimes
does not
make sense)

Backjumping

► Considers the conflict set.
which contains variables that
are responsible of making
one of the possible values
impossible.

→ backtracks to the most
recent assignment in
the conflict set
+ ~~returns the failure indicator~~

← it would jump the problem there

if no legal values found:

the algorithm return the most recent
element in the set along with the
failure indicator.